

University of Pittsburgh

Introduction to Operating Systems
CS 1550

Spring 2022

Sherif Khattab

ksm73@pitt.edu

(Some slides are from **Silberschatz, Galvin and Gagne ©2013)**

Announcements

- Upcoming deadlines:
  - Homework 10: due on 4/11
  - Project 3: due on 4/11
  - Lab 4: due on 4/15

Previous lecture …

- How to allocate disk blocks to files and directories?
  - contiguous and linked

Muddiest Points (Memory)

- Is the reference bit initially set to 0 or 1 after a page is loaded into memory following a page fault?
- Page locking
- Unclear on how to use valid bit to indicate page fault. Is that because valid bit == 0 means the data at this address is invalid, so we need to fetch a valid data from disk?

Muddiest Points (File System)

- Not sure what the numbers in the FAT cells represent?
- Is it better to the int next_block at the beginning or end of the structure?
- offset calculation was a little quick, can we review it
- Logical to physical mapping calculation
- Are block sizes all the same when using linked allocation?
- What allocation method is used most commonly in modern OS's?

Muddiest Points (Project 3)

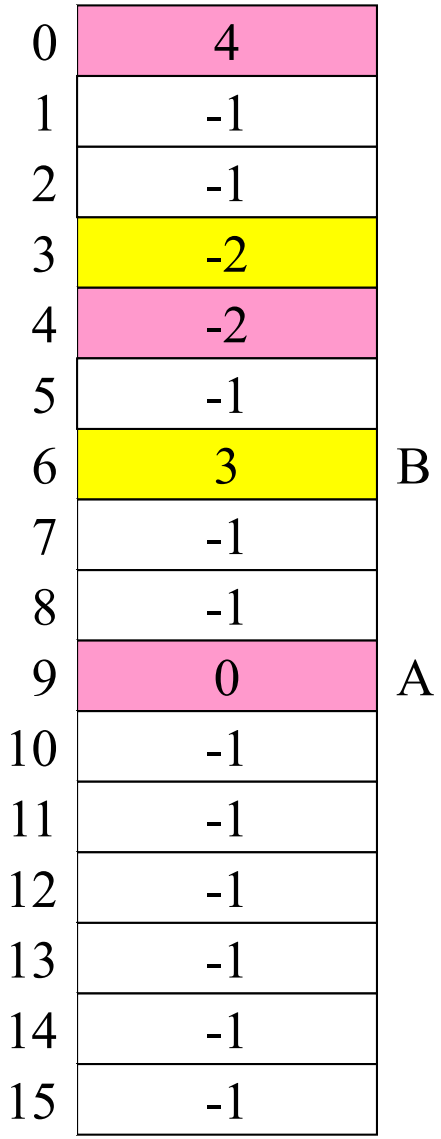
- Would you please explain about project 3?

Problem of the Day

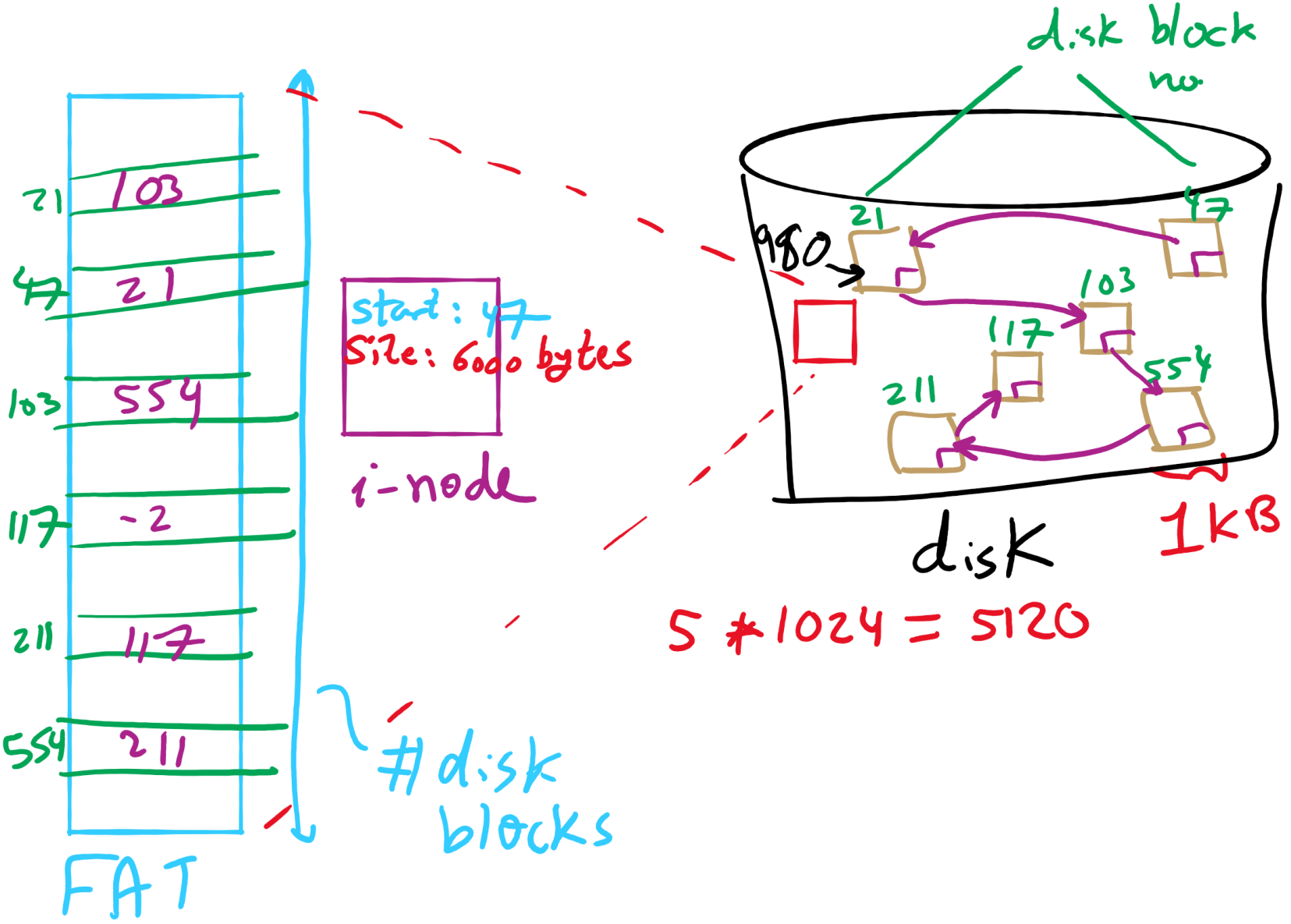
How to allocate disk blocks to files and directories?

File Allocation Table (FAT)

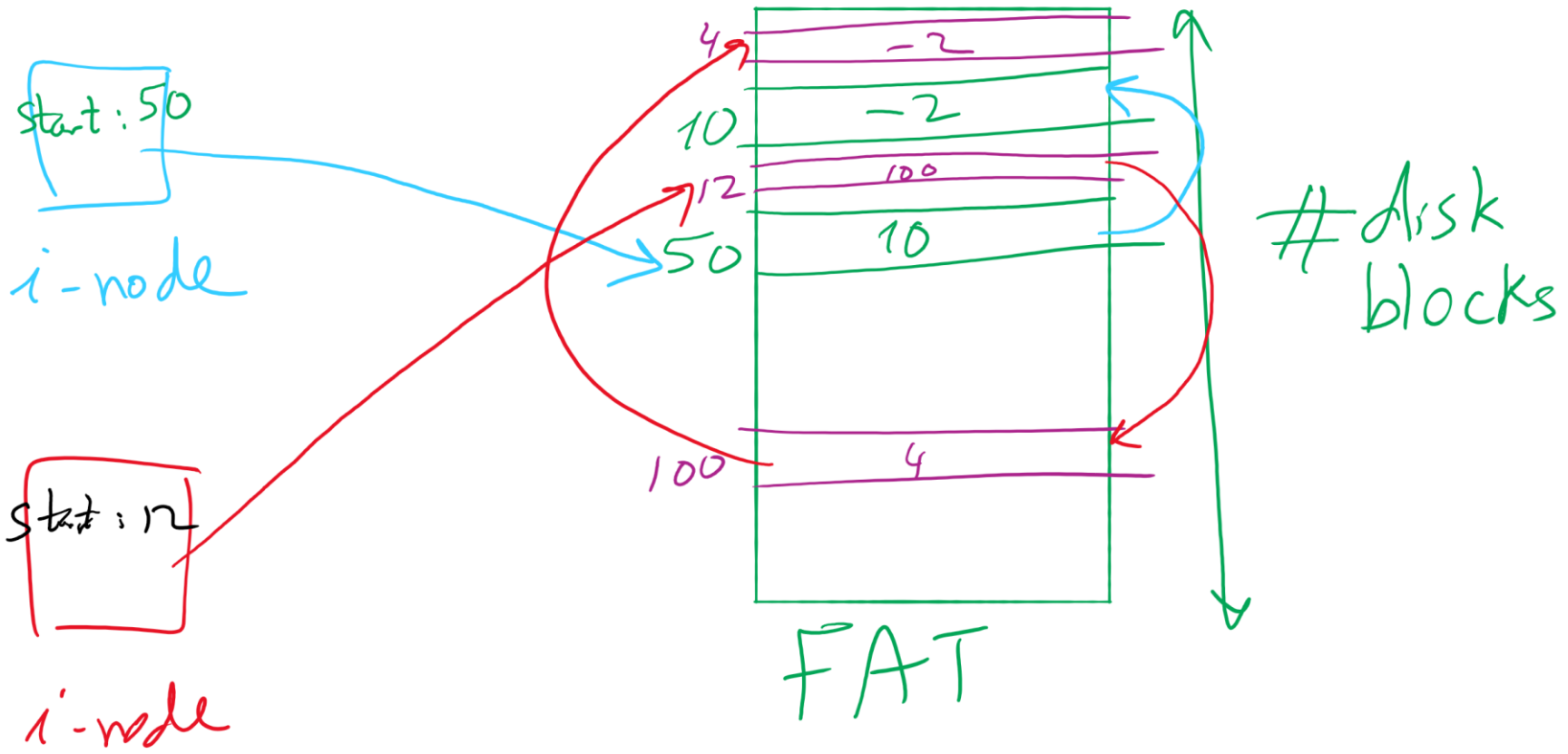
- Links on disk are slow
- Keep linked list in memory
- Advantage: faster
- Disadvantages
  - Have to copy it to disk at some point
  - Have to keep in-memory and on-disk copy consistent



FAT Example

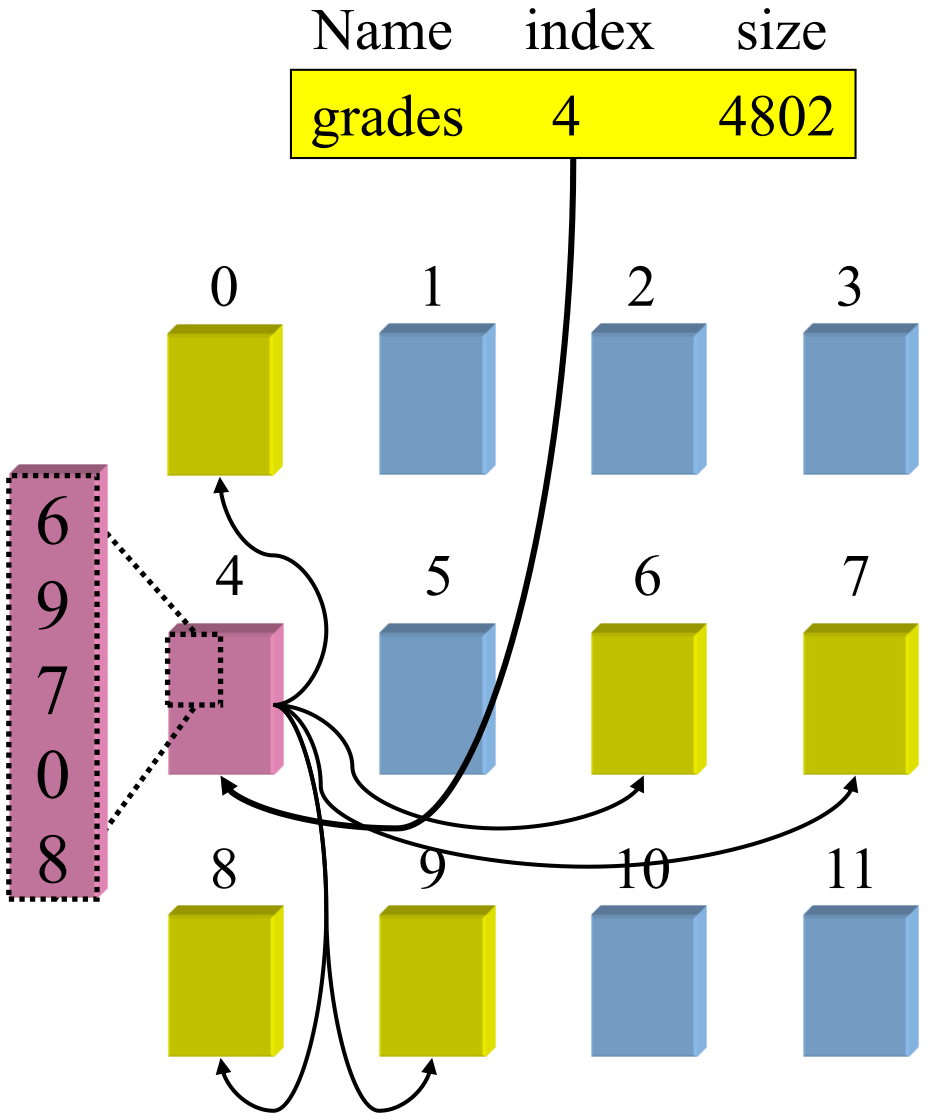

disk block no
21
103
47
21
103
554
117
-2
211
117
554
211
FAT
start: 47
size: 6000 bytes
i-node
disk blocks
980
21
47
103
117
554
211
disk
1kB
5 * 1024 = 5120

i-node and FAT

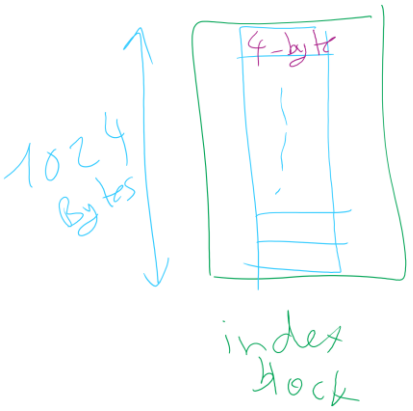


Using a block index for allocation

- Store file block addresses in an array
  - Array itself is stored in a disk block
  - Directory has a pointer to this disk block
  - Non-existent blocks indicated by -1
- Random access easy
- Limit on file size?



Max File Size for Indexed Allocation



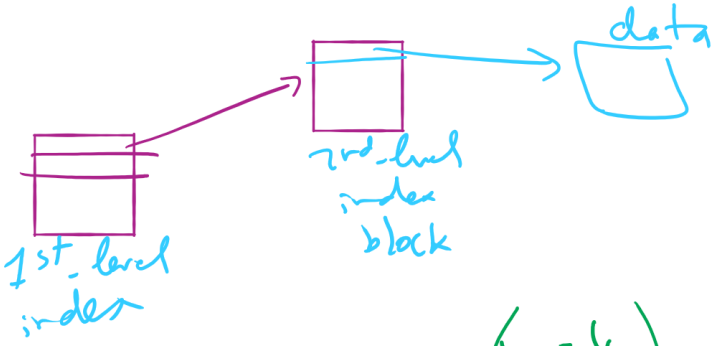
disk block size

$$\frac{1024}{4} = 256$$

block number size

$$\text{Max File Size} = \frac{1024}{4} * 1024$$

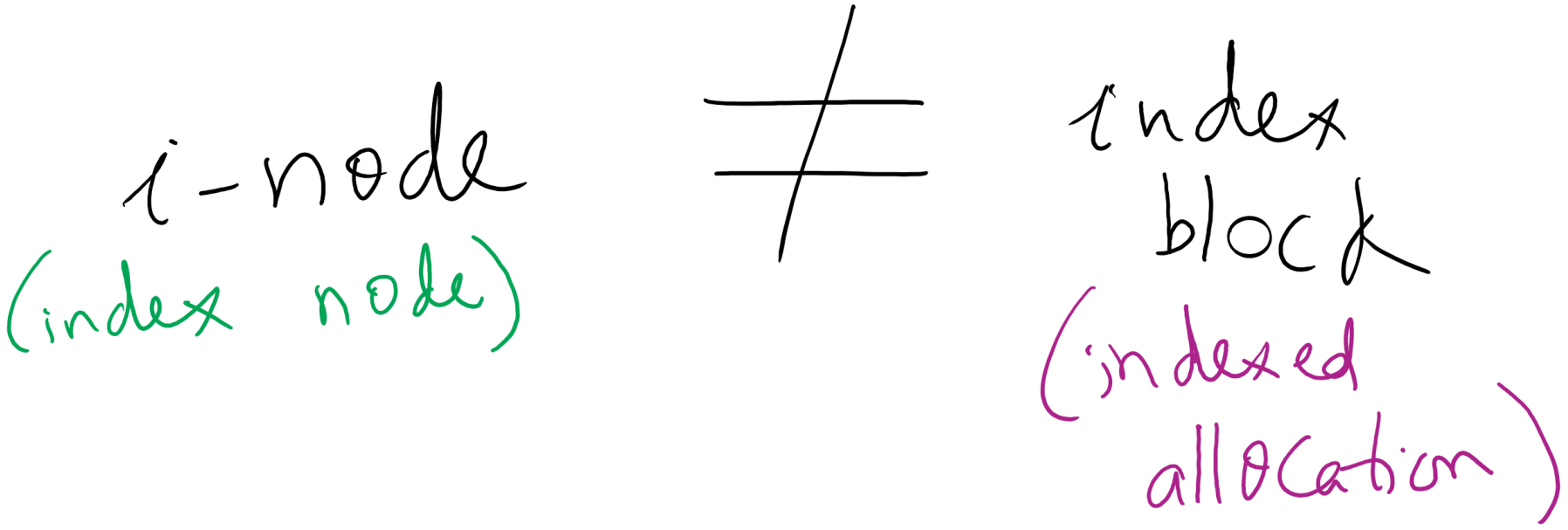
$$= \frac{(1024)^2}{4}$$



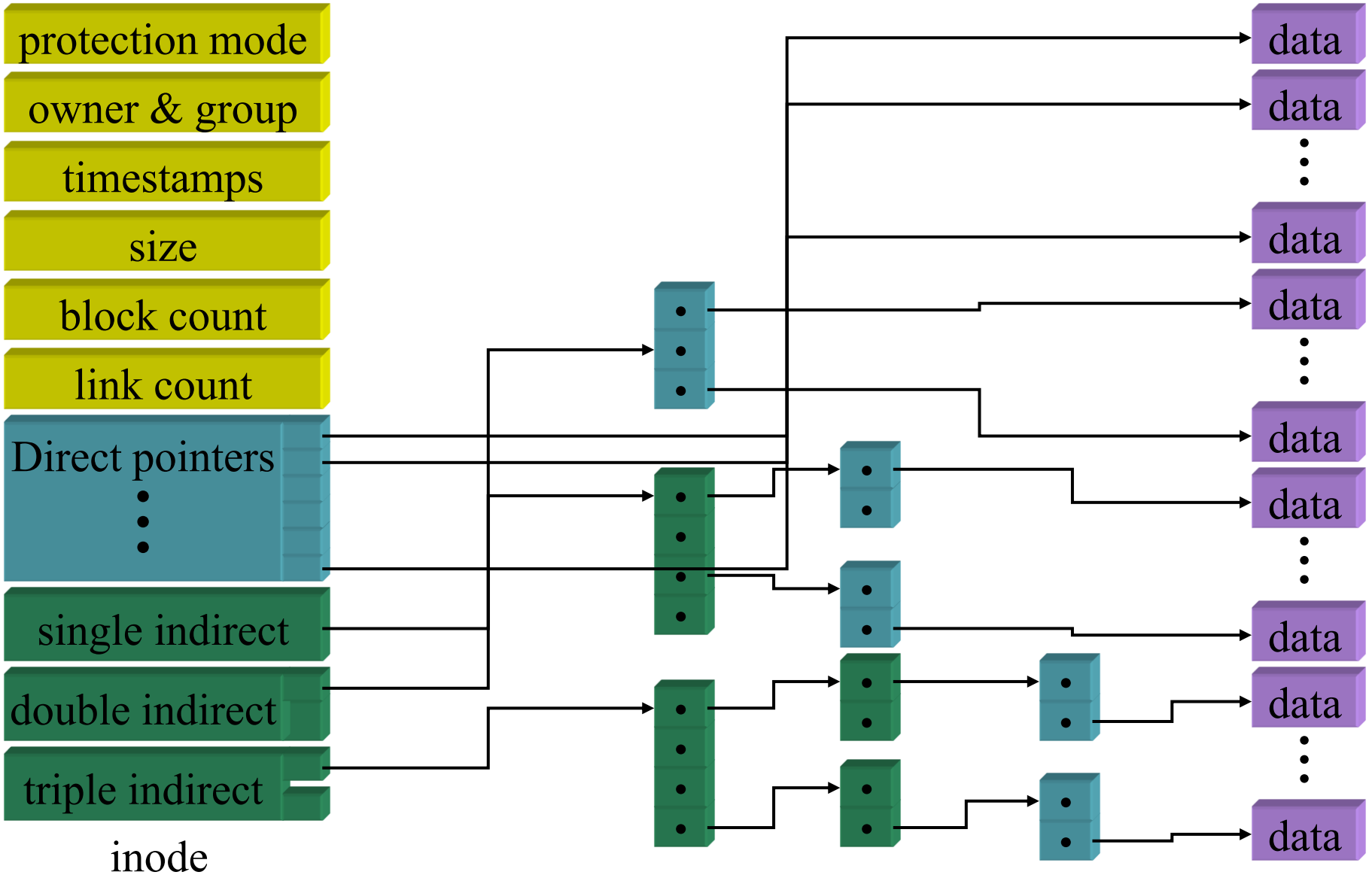
$$\text{Max File Size} = \left(\frac{1024}{4}\right) * \left(\frac{1024}{4}\right) * 1024$$

$$= \frac{(1024)^3}{4^2}$$

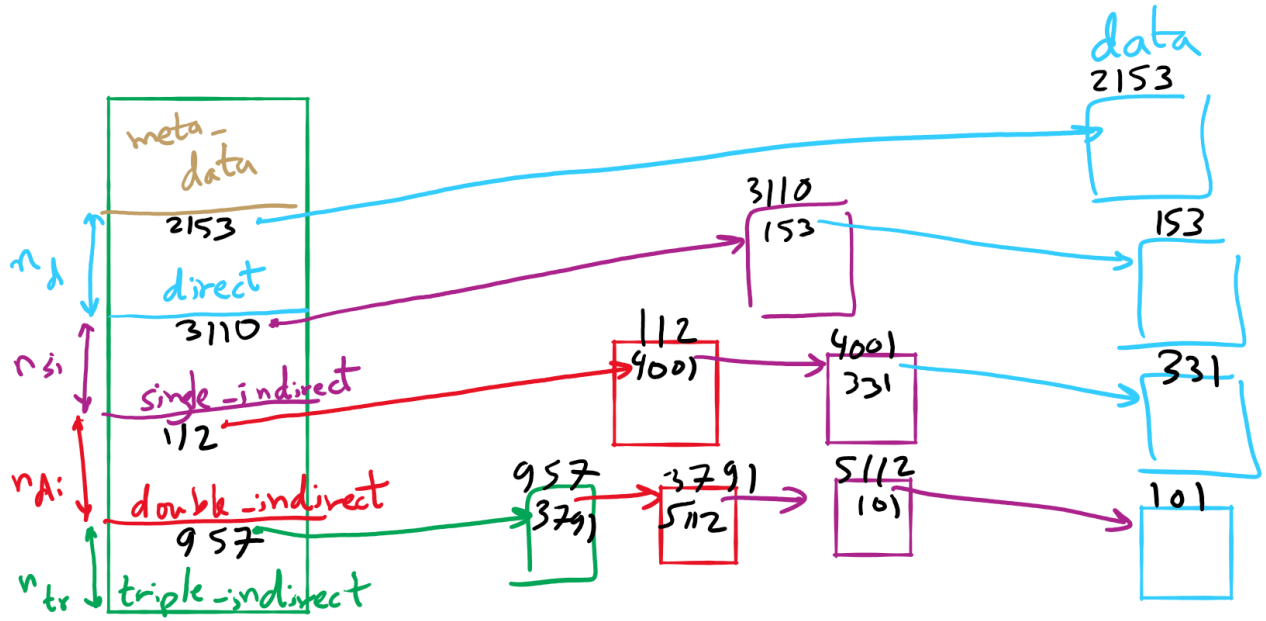
i-node vs index block



Unix Fast File System indexing scheme



FFS and Max File Size



Max File size = i-node

$+ n_d * \text{block size}$

$+ n_{si} * \frac{\text{block size}}{\text{block no. size}} * \text{block size}$

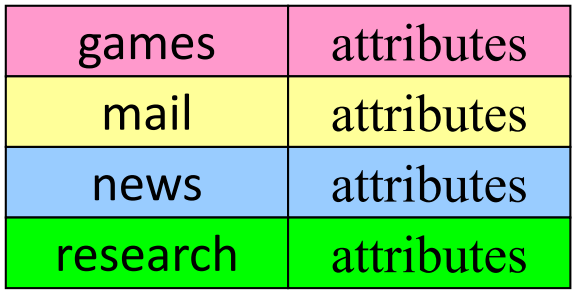
($\frac{\text{block size}}{\text{block no. size}}$ = # entries in an index block)

$+ n_{di} * \left(\frac{\text{block size}}{\text{block no. size}}\right) * \left(\frac{\text{block size}}{\text{block no. size}}\right) * \text{block size}$

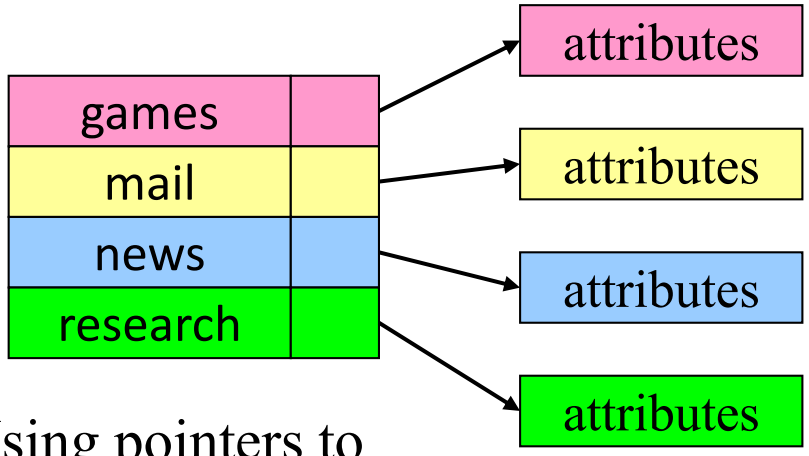
$+ n_{tr} * \left(\frac{\text{block size}}{\text{block no. size}}\right)^3 * \text{block size}$

What's in a directory?

- Two types of information
  - File names
  - File metadata (size, timestamps, etc.)
- Basic choices for directory information
  - Store all information in directory
    - Fixed size entries
    - Disk addresses and attributes in directory entry
  - Store names & pointers to index nodes (i-nodes)

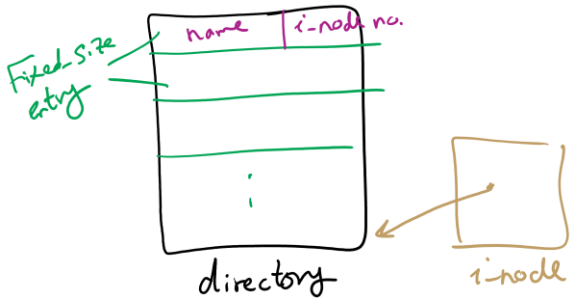


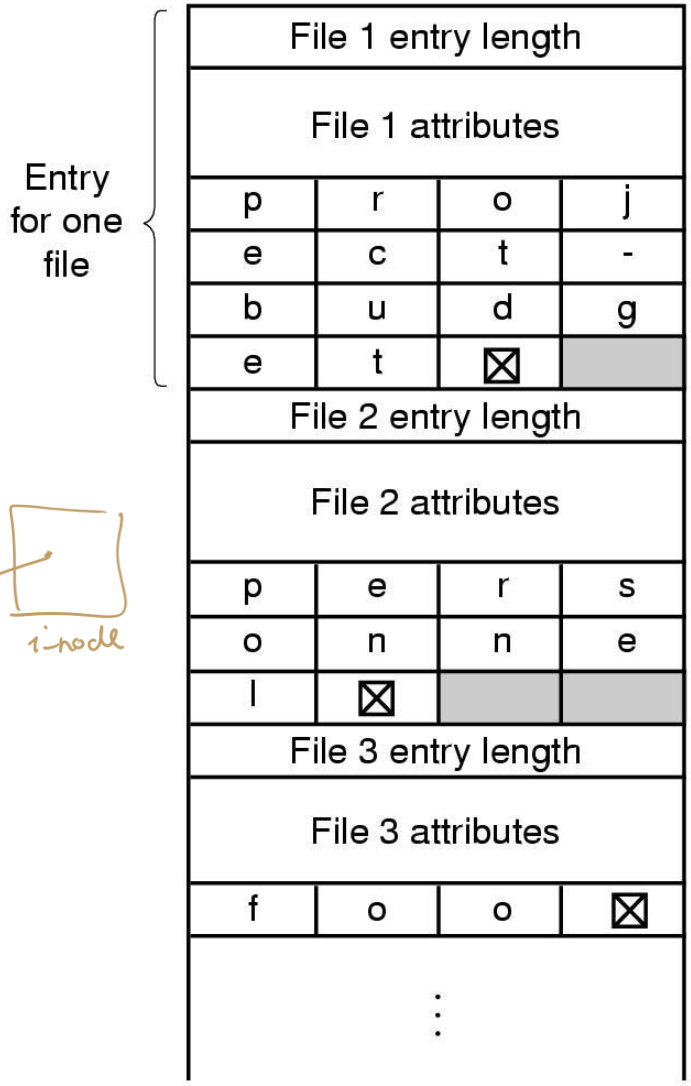
Storing all information in the directory



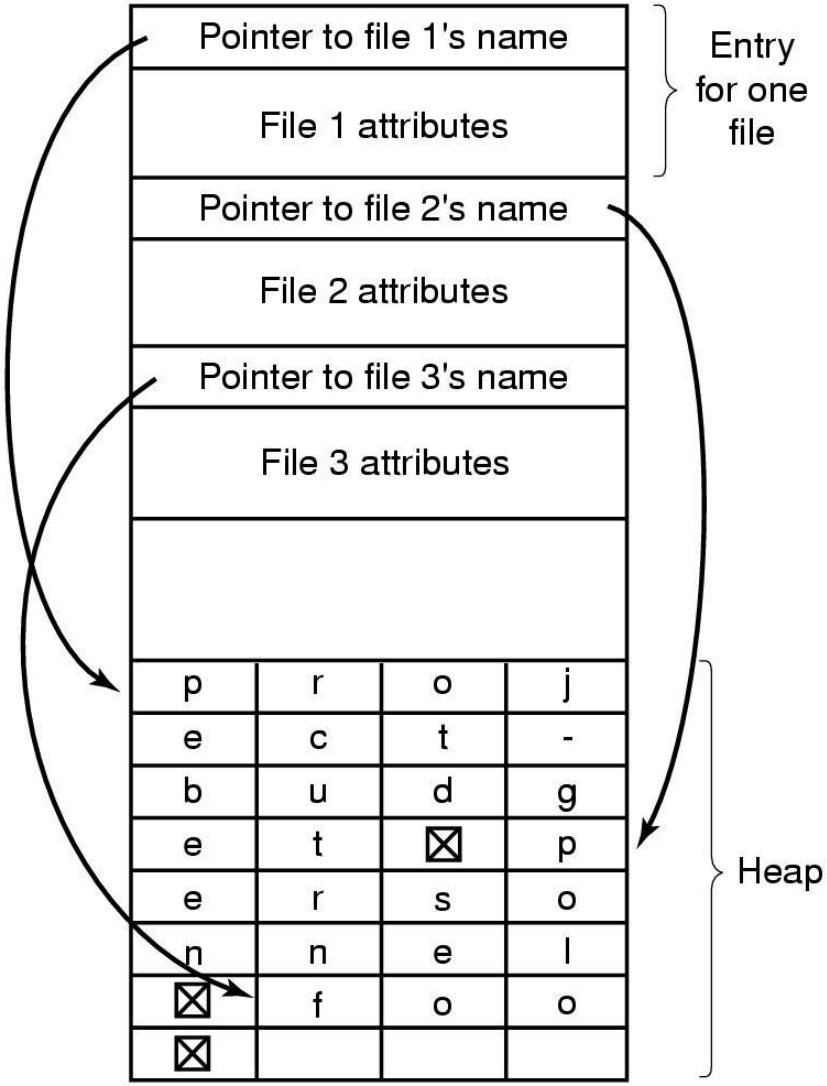
Using pointers to index nodes

Handling long file names in a directory



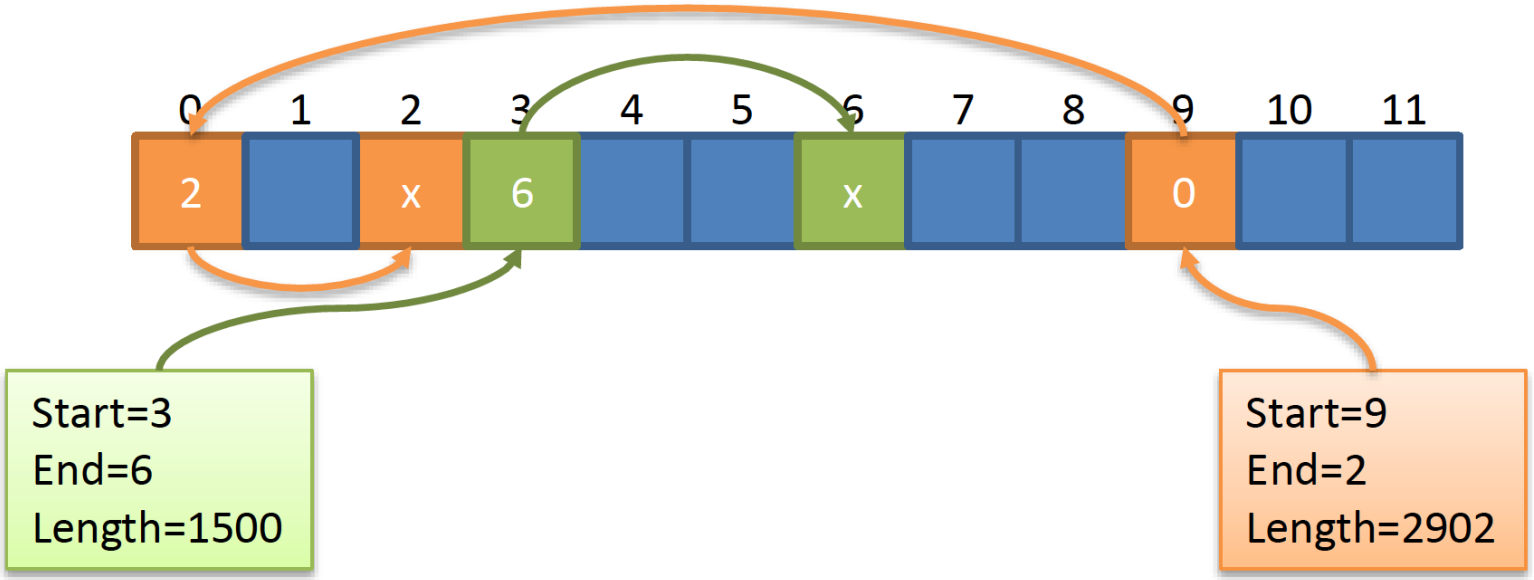


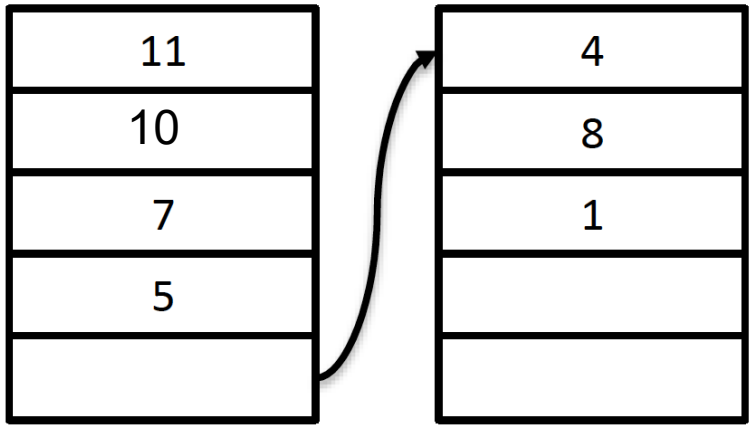
(a)



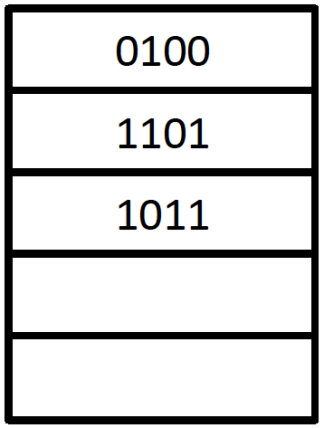
(b)

Free Block Tracking



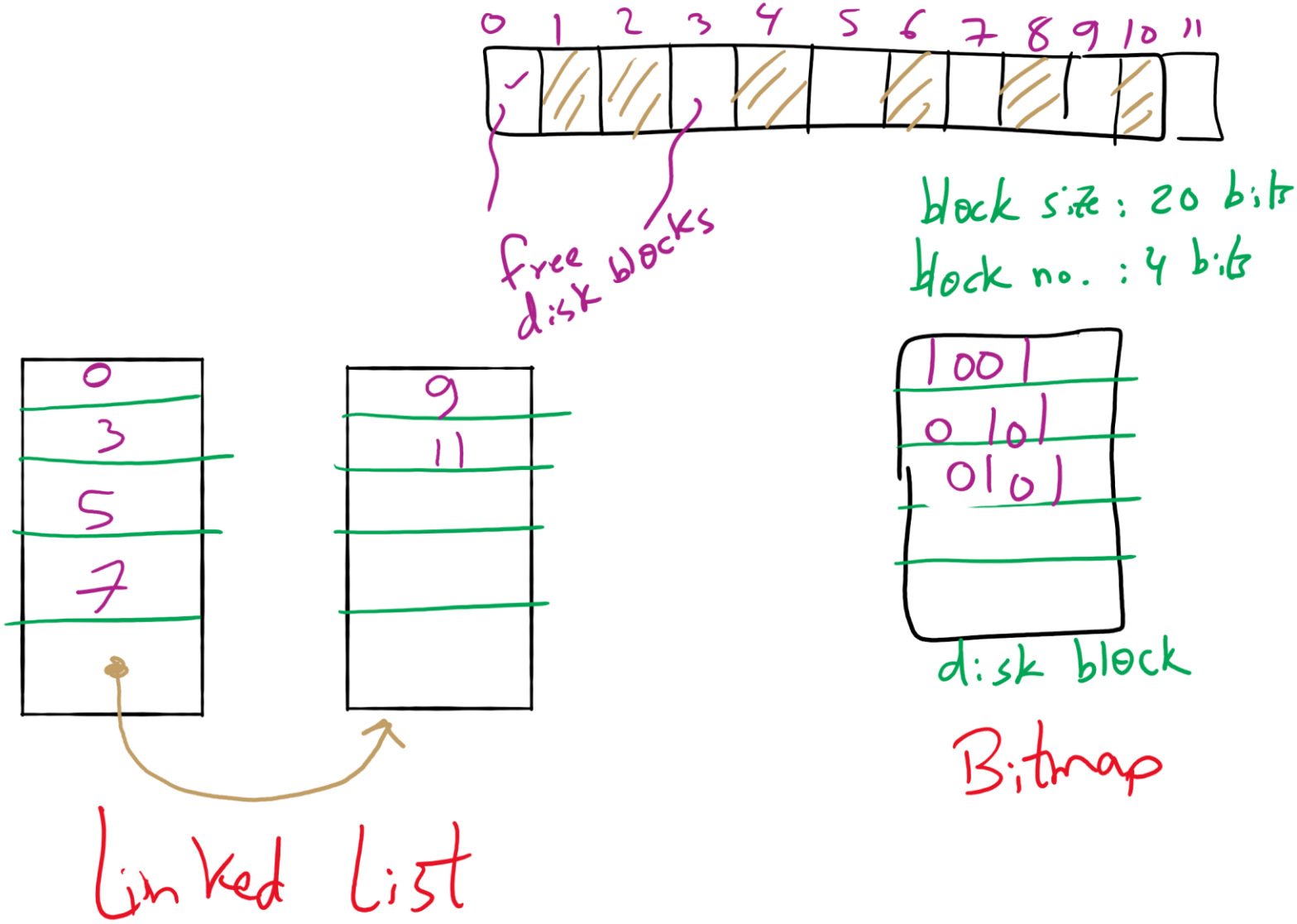


Linked List



Bitmap

Free Block Tracking Example 1



Free Block Tracking Example 2



20-bit disk block

disk block no. : 4 bits

